Favoritar e desfavoritar produtos - CARDÁPIO

Etapa 1— Acrescentar imports

No início de src/app/cardapio.tsx, ajuste os imports existentes e acrescente AsyncStorage:

```
import { useState, useEffect, useRef, useCallback } from "react";
import { useFocusEffect, router, useLocalSearchParams } from "expo-router";
import AsyncStorage from "@react-native-async-storage/async-storage";
```

Mantenha os demais imports. O useFocusEffect atualiza os favoritos quando a tela recebe foco novamente.

Etapa 2 — Criar estados de controle

Dentro de CardapioScreen(), abaixo dos estados produtos, categorias e semImagem, acrescente:

```
const [mensagemFavorito, setMensagemFavorito] = useState("");
const [processandoFavoritos, setProcessandoFavoritos] = useState<number[]>([]);
const bloqueiosFavoritos = useRef<Set<number>>(new Set());
```

processandoFavoritos desabilita a estrela durante a requisição;
bloqueiosFavoritos impede dois toques quase simultâneos antes da atualização do estado.

Etapa 3 — Separar o carregamento das categorias

Precisamos remover apenas a função carregarProdutos() e sua chamada do useEffects inicial. Mas manteremos o carregamento de categorias como abaixo; ele continua sendo executado uma vez ao montar a tela:

```
useEffect(() => {
  async function carregarCategorias() {
    ...
}, []);
```

Etapa 4 — Carregar produtos e favoritos juntos

Após o useEffect inicial crie esta função. Ela consulta os produtos e, se houver token, busca os favoritos do usuário. A união é feita pelo id_produto.

```
const carregarProdutosEFavoritos = useCallback(async () => {
  try {
    const token = await AsyncStorage.getItem("token");
    const respostaProdutos = await fetch(`${API}/produtos`);
    const jsonProdutos = await respostaProdutos.json();
    if (!respostaProdutos.ok || !jsonProdutos.success ||
        !Array.isArray(jsonProdutos.data)) {
      setMensagemFavorito("Não foi possível carregar os produtos.");
      return;
    }

    let idsFavoritos = new Set<number>();
    if (token) {
      const respostaFavoritos = await fetch(`${API}/favoritos`, {
        headers: {
          Accept: "application/json",
          Authorization: `Bearer ${token}`,
```

```
        },
      });
      const jsonFavoritos = await respostaFavoritos.json();
      if (respostaFavoritos.ok && jsonFavoritos.success &&
          Array.isArray(jsonFavoritos.data)) {
        idsFavoritos = new Set<number>(
          jsonFavoritos.data.map((item: any) => Number(item.id_produto))
        );
      } else {
        setMensagemFavorito("Não foi possível carregar seus favoritos.");
      }
    }

    const produtosAtivos = jsonProdutos.data
      .filter((produto: any) => produto.status_produto === "ATIVO")
      .map((produto: any) => ({
        ...produto,
        favorito: idsFavoritos.has(Number(produto.id_produto)),
      }))
      .sort((a: any, b: any) =>
        a.nome_produto.localeCompare(b.nome_produto, "pt-BR")
      );
    setProdutos(produtosAtivos);
  } catch (erro) {
    console.log("Erro ao carregar produtos/favoritos:", erro);
    setMensagemFavorito("Não foi possível conectar ao servidor.");
  }
}, []);
```

A ordenação foi ajustada e coloquei novamente o localeCompare, pois nomes são textos e estamos ordenando por eles.

Etapa 5 — Atualizar ao entrar no cardápio

Logo após carregarProdutosEFavoritos, acrescente:

```
useFocusEffect(
  useCallback(() => {
    carregarProdutosEFavoritos();
  }, [carregarProdutosEFavoritos])
);
```

Isso atualiza as estrelas ao abrir o cardápio e ao voltar de outra tela. O efeito de categorias permanece separado.

Etapa 6 — Alterar a função alterarFavorito(id) pela versão com API

Exclua ou altere a função alterarFavorito(id: number) antiga e coloque esta no mesmo lugar:

```
async function alterarFavorito(id: number) {
  if (bloqueiosFavoritos.current.has(id)) return;
  const produto = produtos.find((item) => item.id_produto === id);
  if (!produto) return;

  const token = await AsyncStorage.getItem("token");
  if (!token) {
    setMensagemFavorito("Faça login para salvar seus favoritos.");
    return;
  }
```

```
  bloqueiosFavoritos.current.add(id);
  setProcessandoFavoritos((atual) => [...atual, id]);
  setMensagemFavorito("");

  try {
    const estavaFavorito = Boolean(produto.favorito);
    const resposta = await fetch(
      estavaFavorito ? `${API}/favoritos/${id}` : `${API}/favoritos`,
      {
        method: estavaFavorito ? "DELETE" : "POST",
        headers: {
          Accept: "application/json",
          Authorization: `Bearer ${token}`,
          ...(!estavaFavorito
            ? { "Content-Type": "application/json" }
            : {}),
        },
        ...(!estavaFavorito
          ? { body: JSON.stringify({ id_produto: id }) }
          : {}),
      }
    );
    const json = await resposta.json();
    if (!resposta.ok || !json.success) {
      setMensagemFavorito(json.message || "Não foi possível atualizar o favorito.");
      return;
    }

    setProdutos((atual) => atual.map((item) =>
      item.id_produto === id
        ? { ...item, favorito: !estavaFavorito }
        : item
    ));
  } catch (erro) {
    console.log("Erro ao alterar favorito:", erro);
    setMensagemFavorito("Não foi possível conectar ao servidor.");
  } finally {
    bloqueiosFavoritos.current.delete(id);
    setProcessandoFavoritos((atual) => atual.filter((item) => item !== id));
  }
}
```

POST envia { id_produto: id }. DELETE envia o ID do produto na URL. A estrela só muda após json.success, evitando mostrar uma alteração que não foi salva.

Etapa 7 — Ajustar o botão da estrela

No trecho que desenha cada produto, mantenha o estilo e o texto da estrela; acrescente disabled e acessibilidade:

```
<Pressable
  style={cardapioStyle.btnFavorito}
  onPress={() => alterarFavorito(produto.id_produto)}
  disabled={processandoFavoritos.includes(produto.id_produto)}
  accessibilityRole="button"
  accessibilityLabel={produto.favorito
    ? "Remover dos favoritos" : "Adicionar aos favoritos"}
>
  <Text style={cardapioStyle.iconeFavorito}>
    {produto.favorito ? "★" : "☆"}
  </Text>
</Pressable>
```

Etapa 8 — Mostrar mensagens ao usuário

Dentro de <View style={cardapioStyle.main}>, antes da busca, acrescente:

```
{mensagemFavorito ? (
  <Text style={{ color: "#B42318", textAlign: "center", marginBottom: 10 }}>
    {mensagemFavorito}
  </Text>
) : null}
```

O estilo está inline para evitar alterações obrigatórias em cardapioStyle.ts. Se preferir, transfira essas propriedades para um estilo mensagemFavorito.

Etapa 9 — Favoritos na página Home

Faça a mesma estrutura descrita nesse documento, com um ajuste específico no carregarProdutosEFavoritos(), referente aso produtos em destaque que são mostrados na tela Home. Ao final do código antes da instrução catch, após o setProdutos(produtosAtivos), acrescente:

```
setProdutosEmDestaque(
      produtosAtivos.filter((produto: any) =>
        produto.destaque_produto === "SIM"
      )
    );
    setMensagemFavorito("");
```